

Rapport TP - Synchronisation des processus, tubes et signaux

Alexis Serrano

1^{er} avril 2025

1 C'est du lourd

1. Combien de processus sont créés par ce programme ?
500
2. Quel est le but de ce programme ?
Le programme créé 500 processus, les affiche et les compte.
3. Que va-t-il se passer lors de l'exécution, c'est-à-dire que va produire l'exécution de ce programme à l'écran ?
Il affiche les fils de 2 à 499, puis affiche le processus père 1.
4. Êtes-vous surpris du résultat de l'exécution du programme ? pourquoi ? expliquez ce qui ne marche pas bien :
La variable somme a la valeur 1 après tous les fils. En effet, le père qui a la somme à 0 appelle 499 fils qui ont chacun le même père et donc incrémente de 1 la somme et se termine, donc la variable n'est pas augmentée.
5. Pourquoi c'est toujours la ligne du processus père qui s'affiche en dernier ?
Car l'affichage du processus père se fait après avoir appelé ses fils.

2 Se sentir plus léger

1. Quelle(s) différence(s) avec le programme précédent ?
Maintenant, les fils renvoient leur valeur de somme pour l'incrémenter. Les threads s'exécutent puis attend que tous les fils se rejoignent pour finir le programme.
2. Prévoyez-vous que son exécution va donner ou pas un résultat différent du programme précédent ? pourquoi ?
Oui, le résultat sera différent : la somme sera incrémentée.
3. Que constatez-vous ? expliquez (sauf si vous aviez deviné juste ci-dessus).
On constate que la somme est bien incrémentée, mais la somme est parfois fausse. En effet, certains fils plus jeunes s'exécutent après un fils plus âgé, ce qui entraîne un retour en arrière.
4. Rappelez quel est l'avantage des threads sur les processus classiques (créés par un appel système `fork()`) :
Les threads permettent une meilleure efficacité et une utilisation réduite des ressources par rapport aux processus classiques, car ils partagent le même espace mémoire et les mêmes ressources. Cela permet de réduire le coût de la création et de la gestion des threads.
5. Mais au fait vous aurait-on menti : vérifions si les processus créés par `fork()` ont bien un numéro de processus différent les uns des autres. Comment afficher le numéro de processus (pid) de chaque processus (et du père aussi) ?
`printf("Pid :%d \n", getpid());`

6. Modifiez `cpt-lourd.c` pour que chaque processus (père et fils) affichent leur pid puis indiquez ce que vous constatez
Les PIDs sont différent dans ce programme.
7. Faites la même modification pour `cpt-leger` et indiquez ce que vous constatez :
Les PIDs sont identiques sur les fils et le père.
8. Dans les deux cas, si vous avez la chance que la commande `pstree` soit installée, lancez-la depuis un autre terminal pendant que chacun des programmes ci-dessus s'exécute, et vérifiez les filiations éventuelles (sinon avec la commande `ps -l`). Ce que vous observez est comme attendu ou pas (pourquoi) ?
9. Vérifions maintenant la charge pour le système d'un programme comparé à l'autre. Après avoir consulté `man time`, expliquez à quoi sert cette commande en deux lignes (max) :
La commande `time` permet de déterminer la durée d'exécution d'une commande donnée.
10. Lancez maintenant les deux programmes précédés de la commande `time`. Quels temps obtenez pour `cpt-leger` en moyenne détaillez user et système :
real 0m0.029s
user 0m0.000s
sys 0m0.037s
11. Idem pour `cpt-lourd` :
real 0m0.040s
user 0m0.074s
sys 0m0.036s
12. Conclusion : quel programme est le plus coûteux pour la machine à faire tourner ? Comment expliquer ça ?
C'est le programme lourd le plus coûteux : cela s'explique par le fait qu'un processus doit copier la mémoire, et cette copie prends du temps.
13. Question

3 Les bons comptes font les bons amis

1. Complétez les phrases suivantes avec les mots proposés en dessous :
*Le problème constaté vient du fait que le programme contient une **section critique** et que l'accès à celle-ci n'est pas protégé. Pour que le programme fonctionne il faut garantir une **exclusion mutuelle**. Ceci peut être réalisé grâce au mécanisme des **sémaphores** qui permet de plus d'éviter l'**attente active** contrairement à d'autres solutions logicielles.*
2. Pour remédier au problème ci-dessus, on va maintenant regarder le programme `mutex-thread.c`. Ouvrez ce programme et remarquez comment en faisant attendre les processus fils un temps non prévisible à un endroit critique, on peut utiliser bien moins de threads pour provoquer le soucis repéré précédemment : compilez et exécutez le programme pour vérifier.
3. On s'aperçoit qu'un mutex est initialisé l 11. Rappelez ce qu'est un mutex, quelle est la différence avec un sémaphore ?
Un mutex (mutual exclusion) est un mécanisme de synchronisation qui permet de garantir qu'une seule ressource (ou section critique) est accessible par un seul thread à la fois. Un sémaphore permet restreindre l'accès à des ressources partagées dans un environnement de programmation concurrente. Le sémaphore permet donc un contrôle plus flexible du nombre de threads pouvant accéder simultanément à la ressource

4. Dans quel cas voudrait-on donner une valeur plus importante à un sémaphore ?
On voudrait donner une valeur plus importante à un sémaphore lorsque l'on souhaite permettre à plusieurs threads d'accéder simultanément à une ressource partagée
5. Placez les instructions suivantes au bon endroit dans le code (à nouveau voir diapos du cours, si nécessaire) :

```
pthread_mutex_lock(&mutex);  
pthread_mutex_unlock(&mutex);
```

Puis compiler et exécuter le code pour voir si le soucis de compteur est réglé ou pas (vérifiez sur plusieurs exécutions). Vérifiez déjà sur plusieurs exécutions du programme que le soucis est réglé, puis pour vérifier d'une autre façon, commentez la partie qui fait dormir les processus et passez à 10 000 processus. Est-ce ok au bout de 10 essais à 10 000 processus ?
C'est
6. Question

4 Signaux et tube

1. Question

5 Le tout à la fois

1. Question